Projeto Integrador: Comunicação e Consistência do Sistema

Monitoramento e proteção por sobrecorrente de um moinho industrial

Fluxo implementado: nó de medição → MQTT → serviço supervisório → banco de dados → dashboard

Grupo

Fernando Grahl Zuffo
Gian Marcos Boaretto
Felipe Castilhos
Luiz Felipe Kauê da Silva

Código, documentação e evidências:
https://github.com/GianMarcosBoaretto/moinho-mqtt

Outubro de 2026

Sumário

1. Resumo da entrega
2. Arquitetura do fluxo
3. Nomeação
4. Sincronização
5. Comportamento em falha de comunicação
6. Decisões e justificativas
7. Evidências de funcionamento

1. Resumo da entrega

Resumo para anexar à documentação do Projeto Integrador. Cada item do enunciado aponta para onde está a resposta completa neste repositório.

Projeto: monitoramento e proteção por sobrecorrente de um moinho industrial. **Grupo:** Fernando Grahl Zuffo, Gian Marcos Boaretto, Felipe Castilhos e Luiz Felipe Kauê da Silva.

Fluxo escolhido

```
nó de medição ──MQTT (QoS 1)──▶ broker Mosquitto ──MQTT──▶ serviço supervisório ──▶ SQLite ──▶ dashboard
  (ESP32, simulador                                          (FastAPI)                          (navegador)
   ou celular)
     origem                comunicação                  componente que recebe          ação: o operador vê
```

O nó **mede, decide e desarma localmente**: a proteção não depende da rede. O fluxo implementado é a camada de monitoramento. O dado é **usado** pelo destino: o supervisório valida, deduplica, grava e mostra corrente, estado, alarmes e histórico. O dashboard também avisa quando o dado está desatualizado.

O que foi pedido × onde está

#	Pedido	Resposta curta	Detalhe
1	Arquitetura atualizada, com componentes e comunicações	diagrama com nó, broker, supervisório, banco, navegador e celular; protocolo de cada enlace e quem é cliente/servidor; diagrama de sequência da falha	**seção 2** (Arquitetura do fluxo)
2	Fluxo funcionando, com evidência de envio, recebimento e uso	teste automatizado com 14 verificações, todas aprovadas; logs do nó e do supervisório; JSON da API em cada etapa; prints	**seção 7** (Evidências de funcionamento), pasta `evidencias/`
3	Nomeação	tópicos `fabrica/<planta>/<device_id>/<tipo>` ; `client_id` único por conexão; `device_id` igual no tópico, no payload e na chave do banco; payload JSON com `schema` versionado	**seção 3** (Nomeação)
3	Sincronização	`ts_origem` gerado na medição (NTP no ESP32); backend grava `ts_recebimento` e o atraso (skew); tudo ordenado por `ts_origem` ; `seq` monotônico para deduplicar e achar lacunas	**seção 4** (Sincronização)
4	Dados durante a falha	proteção segue local; telemetria vai para um buffer circular de 300 amostras (5 min); acima disso perde a mais antiga e a lacuna fica visível	**seção 5** (Comportamento em falha de comunicação §1)
4	O que o usuário vê	banner "Comunicação perdida há Xs — último dado recebido às HH:MM:SS"; valores em cinza com "VALOR ANTIGO"; faixa "sem dados" no gráfico; causa provável	**seção 5** (Comportamento em falha de comunicação §2)
4	Quando a comunicação volta	buffer reenviado em ordem com `seq` / `ts_origem` originais; duplicados descartados pela chave `(device_id, seq)` ; histórico reconstruído no horário certo	**seção 5** (Comportamento em falha de comunicação §3)

#	Pedido	Resposta curta	Detalhe
4	Como se identifica dado desatualizado	STALE quando `agora − ts_origem > 5 s`, independente de quando o dado chegou; LWT como complemento	**seção 5** (Comportamento em falha de comunicação §4)
—	Replicação e consistência	buffer do nó, sessão persistente, status retained e banco como cópias; replicação assíncrona; convergência por identidade única + escrita idempotente + ordem por `ts_origem` = consistência eventual	**seção 6** (Decisões e justificativas)
—	Decisões e justificativas	proteção local, MQTT com QoS 1, consistência eventual, ordenação por origem, tecnologias, ponte do celular	**seção 6** (Decisões e justificativas)
—	Código	simulador, firmware ESP32, backend, dashboard, página do celular, scripts de teste	este repositório (`README`)

Resultado do teste de falha (14/10/2026)

Execução de `scripts/teste_falha_local.ps1` (Windows, sem Docker), com o broker derrubado por 32 s:

Verificação	Resultado
Comunicação normal: dado atual, leituras gravadas	✅
Broker fora + sobrecorrente: **o nó desarmou localmente, sem rede**	✅
Supervisório marcou o dado como STALE e sabe que perdeu o broker	✅
Supervisório ainda mostra o último estado conhecido (OPERANDO), em cinza — o desarme não tinha chegado	✅
Broker de volta: 31 mensagens do buffer reenviadas	✅
Desarme registrado com o **horário em que aconteceu** (durante a falha)	✅
Nenhuma lacuna de `seq` ; histórico ordenado por `ts_origem`	✅
2 duplicados enviados de propósito, 2 descartados	✅

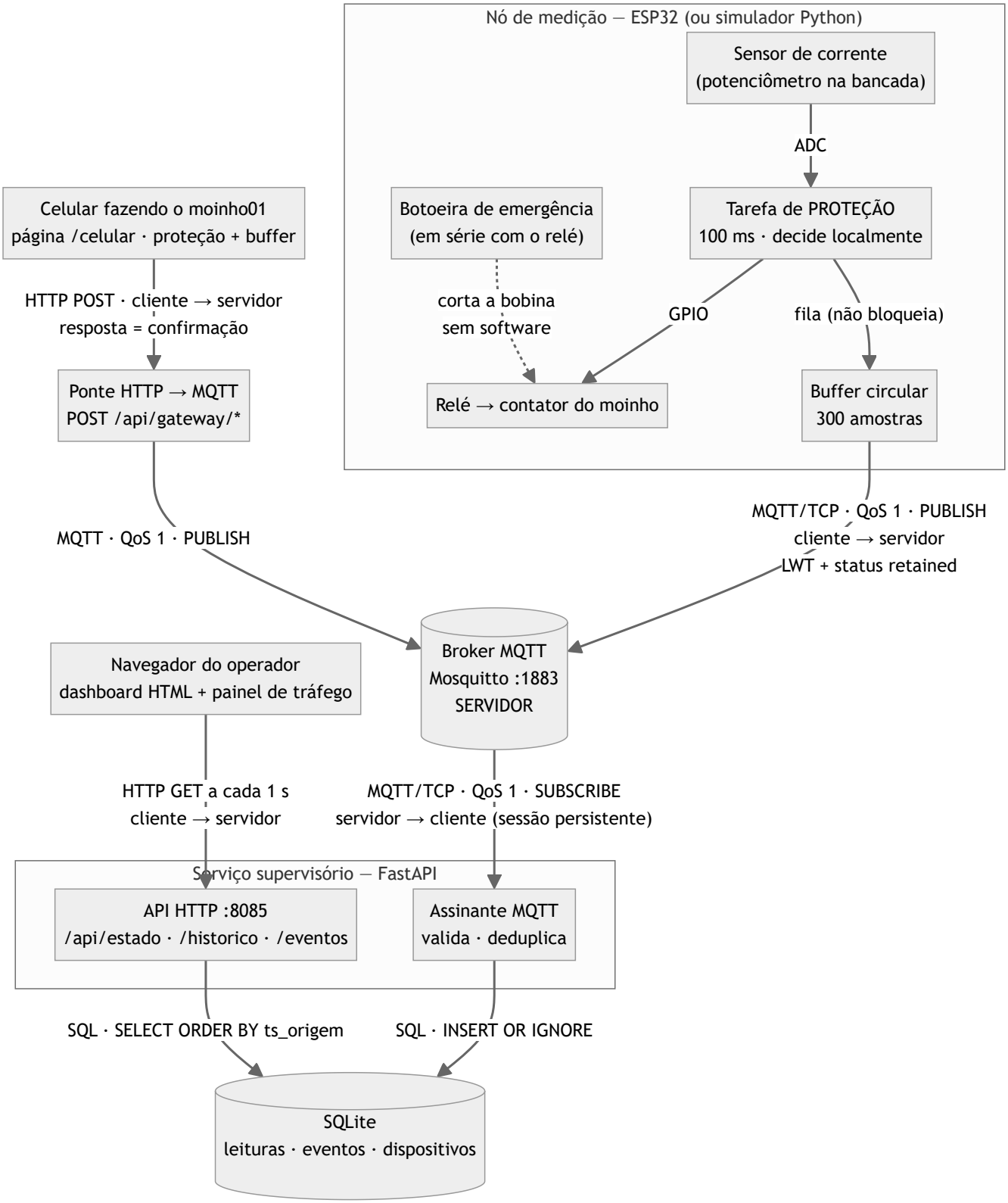
Logs completos: `evidencias/teste-falha-local-2026-10-14/` .

Base para as próximas etapas

O contrato (tópicos com `device_id` , `schema` versionado, `seq` com bloco de boot) já permite acrescentar mais moinhos, comandos remotos e autenticação/TLS sem refazer o fluxo. O que ficou fora desta entrega está listado em **seção 6** (Decisões e justificativas).

2. Arquitetura do fluxo

Componentes, protocolos e papéis

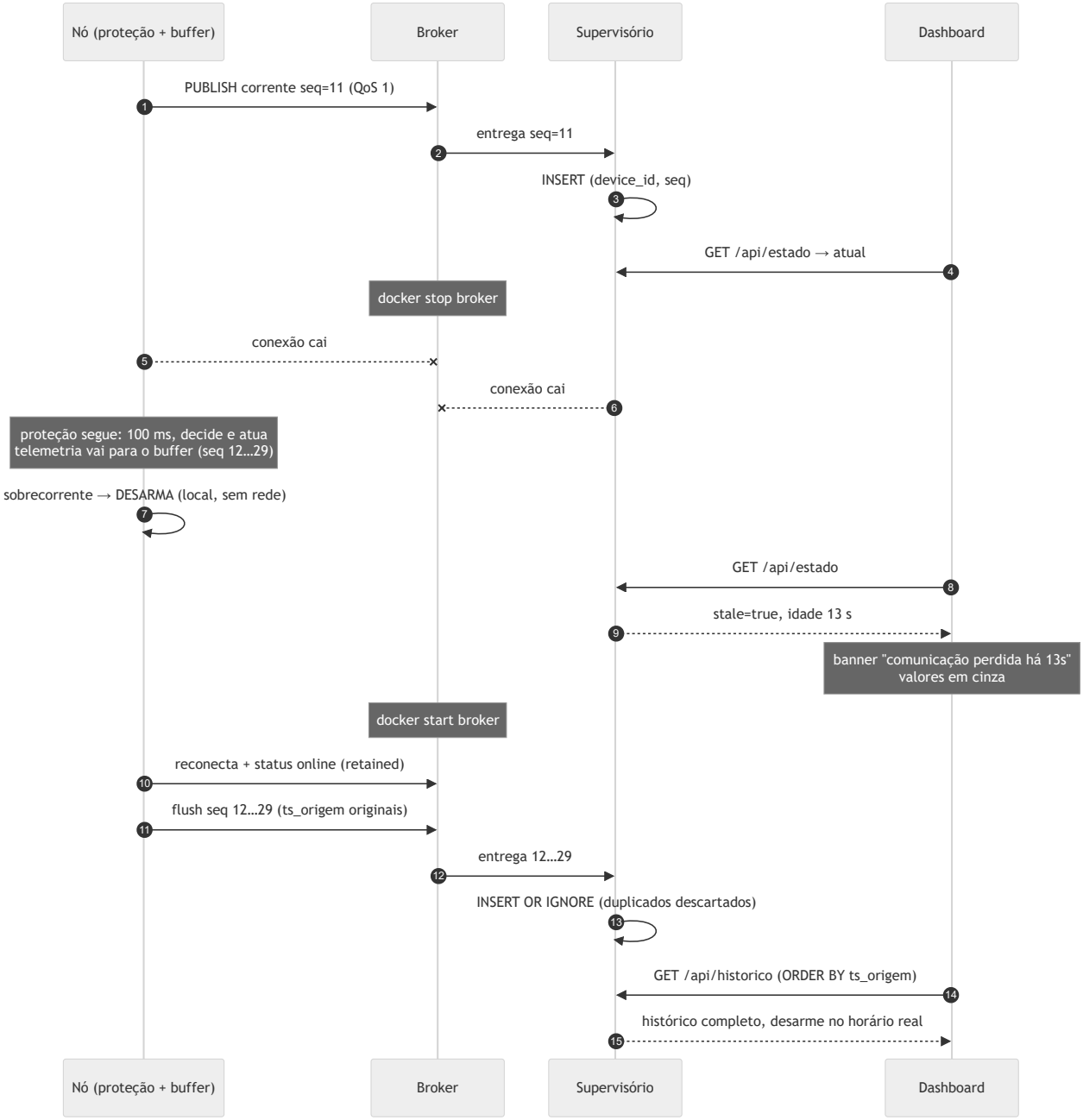


Quem é cliente e quem é servidor em cada enlace

Enlace	Protocolo	Cliente	Servidor	Observação
Nó → broker	MQTT 3.1.1 sobre TCP 1883	nó (`esp32-moinho01` / `sim-moinho01`)	Mosquitto	o nó só **publica**; abre a conexão de saída
Backend ↔ broker	MQTT 3.1.1 sobre TCP 1883	backend (`supervisorio-planta1`)	Mosquitto	o backend **assina** tudo (o broker empurra as mensagens) e **publica** só pela ponte do celular
Backend → banco	SQL (biblioteca embutida)	backend	— (SQLite é um arquivo, não um servidor)	escrita pela thread MQTT, leitura pelas rotas HTTP
Navegador → backend	HTTP/1.1 sobre TCP 8085	navegador	backend (uvicorn)	polling de 1 s
Celular → backend (ponte)	HTTP/1.1 sobre TCP 8085	navegador do celular	backend	o backend republica no MQTT com QoS 1
Sensor/botoeira → relé	elétrico	—	—	**fora da rede**: é o caminho de segurança

No MQTT, publicador e assinante são **os dois clientes**; o único servidor é o broker. Por isso o nó e o backend não se conhecem e podem cair e voltar de forma independente.

Sequência de uma falha de comunicação



3. Nomeação

Todo dado do sistema é identificado da mesma forma em três lugares — tópico MQTT, payload e banco — para que nunca haja dúvida sobre **de quem** é uma leitura.

Identificadores

Nome	Valor no protótipo	Onde aparece	Regra
`planta`	`planta1`	tópico	uma por unidade fabril
`device_id`	`moinho01`	tópico, payload, chave do banco	um por **equipamento monitorado**; nunca muda, mesmo trocando o ESP32
`client_id` MQTT	`esp32-moinho01` (placa) · `sim-moinho01` (simulador) · `supervisorio-planta1` (backend)	conexão MQTT	único e estável por **conexão**; dois clientes com o mesmo id derrubam um ao outro

O celular, quando faz o papel do `moinho01` (página `/celular`), não abre conexão MQTT própria: ele envia por HTTP para `POST /api/gateway/<tipo>`, e o backend publica no tópico dele com a conexão `supervisorio-planta1`. O `device_id` passa por validação (`[a-z0-9_-]`, até 32 caracteres) antes de virar tópico, para que ninguém publique em `#` ou em outro ramo.

Para o `seq` do celular não colidir com o do simulador (mesmo `device_id`), o "boot" do celular é o **minuto atual contado desde 01/01/2026** (centenas de milhares), e não um contador 1, 2, 3... Se outra fonte publicar como `moinho01` ao mesmo tempo, a página do celular mostra um aviso.

O `device_id` identifica o **moinho**; o `client_id` identifica **quem está conectado**. Por isso placa e simulador têm `client_id` diferentes, mas publicam com o mesmo `device_id`: o simulador faz o papel da placa. Os dois nunca devem rodar ao mesmo tempo, porque misturariam duas sequências de `seq` no mesmo `device_id`.

> Visto na prática durante o desenvolvimento: dois simuladores esquecidos rodando com o mesmo `client_id` derrubavam um ao outro a cada ~3 s (o broker desconecta a sessão antiga quando chega uma nova com o mesmo id).

Tópicos

Hierarquia: `fabrica/<planta>/<device_id>/<tipo>`

Tópico	Conteúdo	QoS	Retained	Quando
`fabrica/planta1/moinho01/corrente`	leitura (payload completo)	1	não	a cada 1 s e em toda mudança de estado
`fabrica/planta1/moinho01/alarme`	a leitura que causou a mudança de estado	1	não	desarme, emergência, rearme
`fabrica/planta1/moinho01/status`	conectividade + estado atual	1	**sim**	ao conectar, a cada mudança, e LWT `online:false`

O backend assina com curinga de um nível (`fabrica/+/+/corrente` etc.), então um `moinho02` entra no sistema sem mudar o backend. Ele confere que o `device_id` do payload é igual ao do tópico e descarta a mensagem se não for.

Payload de leitura (`/corrente` e `/alarme`) — schema 1

```
{
  "schema": 1,
  "device_id": "moinho01",
  "seq": 1000000123,
  "ts_origem": "2026-10-13T12:29:22.820Z",
  "corrente_a": 72.4,
  "estado": "operando",
  "motivo": null
}
```

Campo	Tipo	Regra
`schema`	inteiro	versão do contrato. O backend só aceita `1`; mudança incompatível = `2`
`device_id`	texto	igual ao do tópico
`seq`	inteiro 64 bits	monotônico por dispositivo; ver abaixo
`ts_origem`	texto ISO-8601 UTC com ms	instante da **medição**, no relógio do nó
`corrente_a`	número ou `null`	ampères, 1 casa; `null` = leitura inválida (falha de sensor)
`estado`	`operando` | `desarmado` | `emergencia`	estado da proteção local
`motivo`	`null` | `sobrecorrente` | `falha_sensor`	por que desarmou (fica travado até o rearme)

A mensagem em `/alarme` é **a mesma** leitura publicada em `/corrente`, com o mesmo `seq`. Assim o alarme não "gasta" um número da sequência e a detecção de lacunas olha uma sequência só.

Payload de status (`/status`, retained)

```
{"schema": 1, "device_id": "moinho01", "online": true,
 "estado": "desarmado", "motivo": "sobrecorrente", "ts_origem": "2026-10-13T12:29:22.820Z"}
```

LWT (publicado pelo broker quando o nó some sem se despedir):

```
{"schema": 1, "device_id": "moinho01", "online": false}
```

`seq` e reinício do dispositivo

`seq = boot × 1.000.000.000 + contador`

O número do boot fica gravado (NVS no ESP32, `estado/boot.json` no simulador) e sobe a cada reinício. Sem isso, o nó voltaria a publicar `seq = 1` depois de um reset e o backend descartaria todos os dados novos como "duplicados" pela chave `(device_id, seq)`. Um salto de bloco (boot 1 → boot 2) **não** é contado como lacuna.

Banco (SQLite)

Tabela	Chave de unicidade	Conteúdo
`leituras`	`UNIQUE(device_id, seq)`	uma linha por leitura, com `ts_origem`, `ts_recebimento` e `skew_ms`

Tabela	Chave de unicidade	Conteúdo
`eventos`	`UNIQUE(device_id, seq)`	desarme/emergência/rearme (com `seq`) e online/offline (sem `seq`)
`dispositivos`	`device_id`	último status de conectividade

4. Sincronização

Dois relógios, dois carimbos

Carimbo	Quem gera	Significa
`ts_origem`	o nó (ESP32 com NTP; simulador com o relógio UTC da máquina)	quando a corrente **foi medida**
`ts_recebimento`	o backend, ao receber a mensagem	quando o dado **chegou** ao supervisório
`skew_ms`	o backend: `ts_recebimento − ts_origem`	atraso da entrega (rede + buffer)

Em operação normal o `skew_ms` fica em poucos milissegundos. Depois de uma falha de comunicação, as leituras que vieram do buffer chegam com `skew_ms` de vários segundos — é assim que o dashboard sabe pintá-las de laranja ("reconstruído do buffer").

Um `skew_ms` **negativo** ou grande demais com a rede normal indica relógio do nó errado (NTP não sincronizou). O valor fica gravado em toda leitura justamente para isso aparecer.

Ordem: sempre por `ts_origem`

O backend **nunca** usa a ordem de chegada:

- histórico: `ORDER BY ts_origem`;
- "leitura atual": a de **maior** `ts_origem`, não a última que chegou;
- eventos: `ORDER BY ts_origem`.

Por que importa: quando a comunicação volta, o buffer despeja de uma vez leituras de minutos atrás, misturadas com leituras novas. Se a tela mostrasse "a última que chegou", o operador veria um valor velho como se fosse o atual.

`ts_origem` é gravado sempre no mesmo formato (`AAAA-MM-DDTHH:MM:SS.mmmZ`, UTC), então a ordenação de texto no SQLite é também a ordenação cronológica.

`seq`: deduplicação e lacunas

O `seq` é monotônico por dispositivo (ver **seção 3** (Nomeação)) e tem dois usos:

1. **Deduplicar.** QoS 1 garante "pelo menos uma vez": se o PUBACK se perde, o mesmo dado chega duas vezes. `UNIQUE(device_id, seq)` + `INSERT OR IGNORE` faz a segunda cópia ser descartada, e o resultado é "efetivamente uma vez". O contador `duplicadas_descartadas` aparece no dashboard.
2. **Detectar lacunas.** Se depois do `seq` 41 vem o 44, faltam 42 e 43. O backend calcula as lacunas com `LAG(seq)` dentro do mesmo bloco de boot. Lacuna que se fecha sozinha = dado que estava no buffer e chegou depois. Lacuna que não fecha = dado perdido de verdade (buffer estourou).

Como cada lado obtém a hora

Simulador: `datetime.now(timezone.utc)` no instante da leitura.

ESP32: `configTime(0, 0, "a.st1.ntp.br", "pool.ntp.org")`. A amostra guarda o relógio monotônico (`esp_timer_get_time()`, em µs desde o boot) e o `ts_origem` é calculado na hora de publicar:

```
ts_origem = agora_ntp − (agora_monotonico − instante_monotonico_da_amostra)
```

Com isso, amostras feitas **antes** de o NTP responder (por exemplo, o Wi-Fi demorou a subir) recebem o horário certo assim que o relógio sincroniza. Até lá elas esperam no buffer — a proteção não espera nada disso.

Limite conhecido

O STALE é calculado com `agora (backend) − ts_origem (nó)`. Se os relógios divergirem, esse cálculo erra pela mesma diferença. Com NTP dos dois lados a diferença é de milissegundos, muito abaixo do limite de 5 s.

5. Comportamento em falha de comunicação

Princípio: **a proteção não depende da rede.** O nó de medição decide e aciona o relé localmente. Wi-Fi, MQTT, backend e banco são só monitoramento. Uma falha de comunicação deixa o operador **sem informação**, mas não deixa o moinho **sem proteção**.

Mecanismos usados

Mecanismo	Onde	Para quê
QoS 1	todas as publicações e assinaturas	entrega "pelo menos uma vez" com confirmação (PUBACK)
Sessão persistente (`clean_session=false`)	nó e backend	o broker guarda as mensagens QoS 1 de quem está desconectado
Persistência do Mosquitto	`mosquitto.conf`	sessões e retained sobrevivem a reinício do broker
LWT `online:false` (retained)	nó	o broker avisa que o nó sumiu sem se despedir
Status retained	nó	quem conecta depois recebe na hora o último status
Buffer circular de 300 amostras	nó	guarda a telemetria enquanto não há broker
`UNIQUE(device_id, seq)`	banco	reenvio não duplica dado
STALE quando `agora − ts_origem > 5 s`	backend/dashboard	nunca mostrar dado velho como atual

1. O que acontece com os dados durante a falha

- **No nó:** a tarefa de proteção continua medindo a cada 100 ms, decidindo e acionando o relé, igual a antes. Ela não chama nada de rede. Cada amostra de telemetria (1 por segundo, mais as de mudança de estado) entra no **buffer circular** com seu `seq` e seu `ts_origem` originais. O log mostra isso a cada 5 s:

`[PROTECAO] sem broker ha 10s - protecao local ATIVA: estado=desarmado corrente=0.2 A | buffer 11/300 descartadas=0`

Um desarme no meio da falha acontece normalmente e é registrado no log como "decidido localmente, SEM REDE". - **Capacidade:** 300 amostras = 5 minutos a 1 amostra/s. Se a falha passar disso, a mais **antiga** é descartada (o log avisa) e vira uma lacuna permanente de `seq` , visível no dashboard. Preferimos perder o dado mais velho a perder o mais recente. - **No broker:** se só o backend caiu, o broker guarda as mensagens QoS 1 na sessão persistente dele (até `max_queued_messages`). Se o broker caiu, quem segura os dados é o buffer do nó. - **No banco:** nada é gravado durante a falha. O banco fica temporariamente incompleto e converge quando a comunicação volta.

2. O que o usuário vê

Quando a leitura mais recente passa de 5 s de idade, o dashboard entra em **modo degradado**:

- **banner vermelho:** "⚠ Comunicação perdida há 13s — último dado recebido às 09:29:17";
- **a causa provável**, na linha de baixo:
- "O supervisório não alcança o broker MQTT" (o backend perdeu a conexão), ou
- "O broker informou que o nó saiu da rede (LWT/offline)", ou

- "Nenhuma leitura nova do nó chegou ao supervisório";
- **corrente e estado em cinza**, com "VALOR ANTIGO" ao lado. O valor continua na tela para referência, mas não parece atual;
- **no gráfico**, uma faixa cinza "sem dados" do último ponto até agora;
- o lembrete fixo: "A proteção do moinho não depende desta tela".

Se o próprio backend cair, o navegador não consegue buscar dados e mostra "Supervisório (backend) inacessível", também com os valores em cinza.

3. O que acontece quando a comunicação volta

1. O nó reconecta (tentativa a cada 1–5 s) e publica o status `online` (retained).
2. O buffer é esvaziado **na ordem**, cada mensagem com o `seq` e o `ts_origem` de quando foi medida: `[MQTT] conectado ao broker (ficou fora 17.1s); buffer com 18 mensagens para reenviar [BUFFER] flush: 18 mensagens reenviadas com ts_origem/seq originais (restam 0)`
3. O backend grava cada uma. Pelo `skew_ms` grande, ele sabe que veio do buffer: `[RX] moinho01 seq=1000000016 130.8 A desarmado skew=12827 ms (BUFFER) [ALARME] moinho01 desarme motivo=sobrecorrente em 2026-10-13T12:29:22.820Z [RECUPERADO] moinho01: dado atual de novo (idade 0.5s); lacunas restantes: 0`
4. Como tudo é ordenado por `ts_origem`, o histórico fica **reconstruído no lugar certo**: o gráfico mostra o trecho da falha em laranja, e o desarme aparece na tabela de eventos com o **horário em que aconteceu**, não com o horário em que chegou.
5. Duplicados — um PUBACK perdido no meio do flush, por exemplo — são descartados pela chave `(device_id, seq)` e contados em "duplicadas descartadas".

3b. E quando o nó é o celular?

O celular (página `/celular`, ver **seção 6** (Decisões e justificativas) D5b) segue as mesmas regras:

Situação	O que acontece
Wi-Fi do celular cai, ou botão "🚫 Simular queda"	proteção continua no celular ("decidido no celular, SEM REDE"); leituras vão para o buffer (300); cabeçalho mostra "sem comunicação"
Broker do PC fora, celular com rede	backend responde `503`; o celular mantém a leitura no buffer e tenta de novo a cada 1 s
Comunicação volta	o buffer é reenviado em ordem, com `seq` / `ts_origem` originais; o celular mostra "flush do buffer" e "PC confirmou … lacunas 0"
No PC	o mesmo banner e os mesmos valores em cinza das seções 2 e 4

Diferença: a página do celular **perde o buffer se for recarregada**, porque ele fica na memória do navegador, e não em disco como no ESP32. O que estava lá vira lacuna de `seq`, e o dashboard mostra isso.

4. Como o sistema identifica um dado desatualizado

- **Regra:** um dado é STALE quando `agora − ts_origem > 5 s`. A leitura chega a cada 1 s, então 5 s são 5 leituras seguidas faltando.
- Usa-se o `ts_origem` (quando foi medido), não o `ts_recebimento`. Um dado velho que acaba de chegar do buffer **continua** velho.
- A "leitura atual" é a de maior `ts_origem`. Uma rajada de dados antigos no flush não substitui um dado mais novo.
- O backend calcula isso a cada consulta (`/api/estado` devolve `stale`, `idade_s` e `stale_limite_s`). Uma thread de vigia registra no log as transições `[STALE]` e `[RECUPERADO]`, que servem de evidência.

- O LWT complementa, mas **não substitui**: quando o broker cai (o caso do teste), não há broker para publicar o LWT. Só a idade do dado pega esse caso. Por isso o critério principal é a idade.

6. Decisões e justificativas

D1. A proteção é local (no nó de medição)

Decisão: o ESP32 mede, decide e aciona o relé. A botoeira de emergência fica fisicamente **em série** com o relé.

Por quê: - Sobrecorrente em motor de moinho se resolve em centenas de milissegundos. Uma decisão que dependesse de Wi-Fi → broker → backend → resposta teria latência variável e vários pontos de falha no caminho. - Se a rede cai, uma proteção remota simplesmente deixa de existir. A local continua igual. - A botoeira em série desliga o motor mesmo com o ESP32 travado, porque não passa por software. - **Fail-safe:** sensor inválido (fora da faixa) por 3 leituras seguidas também desarma (`falha_sensor`), porque sem medição não há proteção. Contato da botoeira aberto ou fio solto é lido como emergência.

Como o código garante isso: - No simulador, a decisão vem antes de qualquer chamada de rede no laço, e a rede roda em outra thread (`loop_start`). - No firmware, a proteção é uma tarefa FreeRTOS com prioridade maior que o `loop()` . Se o `loop()` travar tentando conectar no broker, ela continua no horário, e entrega amostras por uma fila com timeout zero: fila cheia perde telemetria, nunca atrasa a proteção.

D2. MQTT para o monitoramento

Por quê MQTT, e não HTTP do ESP32 para o backend: - **Publish/subscribe:** o nó não precisa saber quem consome os dados. Backend, dashboard e um futuro historiador assinam sem mexer no firmware. - **Conexão de saída do nó:** o ESP32 é cliente; não precisa abrir porta nem ter IP fixo. - **Recursos prontos** para o que esta entrega exige: QoS 1, sessão persistente, LWT (detecção de queda do nó) e retained (último status para quem chega depois). - **Leve:** cabeçalho de 2 bytes, bibliotecas maduras para ESP32 e Python.

Por quê QoS 1 e não 0 ou 2: - QoS 0 perderia leituras em qualquer oscilação. - QoS 2 ("exatamente uma vez") custa 4 mensagens por publicação e ainda dependeria do broker guardar estado. A deduplicação por `(device_id, seq)` no banco dá o mesmo resultado mais barato: "pelo menos uma vez" + idempotência = "efetivamente uma vez".

D3. Consistência eventual é aceitável aqui

Durante uma falha, o banco fica **para trás** do que aconteceu no moinho. Isso é aceitável porque:

1. **Nenhuma decisão de segurança lê o banco.** O desarme acontece no nó antes de qualquer dado sair dele. O banco serve para histórico, telas e análise.
2. **O banco converge sozinho.** Quando a comunicação volta, o buffer reenvia tudo com `ts_origem` / `seq` originais, e o histórico fica igual ao que seria sem a falha. A única exceção é uma falha maior que o buffer (5 min), que aparece como lacuna explícita, e não como dado errado.
3. **A tela não mente sobre isso.** Enquanto o banco está atrasado, o dashboard diz que está ("comunicação perdida há Xs", valores em cinza). O risco da consistência eventual seria o operador agir sobre dado velho achando que é atual; o STALE elimina esse risco.

Consistência forte exigiria que o nó esperasse a confirmação do banco antes de seguir. Isso colocaria a rede no caminho da proteção, que é exatamente o que D1 proíbe.

D3b. Replicação: onde o dado existe e como as cópias convergem

Não há banco replicado nem servidor redundante: é um protótipo. Mesmo assim, o mesmo dado existe em mais de um lugar, e o projeto define quem é a fonte e como as cópias se acertam.

Cópia	Onde	O que guarda	Papel
Buffer do nó	RAM do ESP32 / simulador / celular	últimas 300 leituras ainda não confirmadas	**réplica primária** de tudo que ainda não chegou ao banco: enquanto o broker está fora, o dado só existe aqui
Fila da sessão persistente	broker (Mosquitto, com persistência em disco)	mensagens QoS 1 para um assinante desconectado	segura os dados quando o **backend** cai e o broker continua de pé
Status retained	broker	o último status de cada nó	réplica do **estado atual**: quem conecta depois recebe na hora, sem esperar a próxima publicação
Banco SQLite	backend	todo o histórico	**réplica de leitura** para telas e análise; é para onde todas as outras convergem

Modelo: replicação **assíncrona, primário → réplica**, com o nó de medição como primário. O nó nunca espera o banco confirmar para seguir. Ele só tira a leitura do buffer quando ela foi entregue ao MQTT: no ESP32, depois do PUBACK do broker; no simulador, a biblioteca paho guarda as mensagens ainda sem PUBACK e as retransmite ao reconectar.

Como as cópias convergem: 1. Toda leitura tem identidade única e imutável, `(device_id, seq)`, criada na origem. 2. A réplica (banco) aplica as escritas de forma **idempotente** (`INSERT OR IGNORE` com `UNIQUE(device_id, seq)`). Receber a mesma leitura duas vezes, por reenvio QoS 1 ou flush do buffer, dá o mesmo resultado que receber uma vez. 3. A **ordem de aplicação não importa**: a ordem lógica vem de `ts_origem`, não da ordem de chegada (D4). Duas réplicas que recebem as mesmas leituras em ordem diferente ficam iguais. 4. Lacunas de `seq` mostram quando uma réplica está **incompleta**. Se a lacuna se fecha sozinha, era atraso. Se não fecha, houve perda real (buffer estourou), e ela fica visível no dashboard.

Esses três pontos (identidade única, aplicação idempotente e ordem independente da chegada) são o que garante a **consistência eventual** da D3: sem novas leituras, todas as cópias chegam ao mesmo conteúdo.

D4. Ordenação por `ts_origem`, não por chegada

Explicado em **seção 4** (Sincronização). Resumo: a ordem de chegada é um artefato da rede (o flush do buffer chega fora de ordem em relação ao tempo real). O que interessa ao processo é quando a corrente foi medida.

D5. Tecnologias do protótipo

Escolha	Motivo
Mosquitto em Docker	broker de referência, uma linha no compose, persistência nativa
Python + paho-mqtt (simulador)	roda sem hardware, mesma lógica do firmware, fácil de forçar falhas
FastAPI + SQLite (backend)	um arquivo só, sem servidor de banco; `UNIQUE` e funções de janela (`LAG`) bastam
HTML puro + polling de 1 s	sem build nem framework; 1 s é o mesmo período da telemetria
Buffer de 300 amostras	5 min a 1 Hz cobre a maioria das quedas de Wi-Fi/broker; ~10 KB de RAM no ESP32
STALE em 5 s	5 leituras perdidas seguidas; folga para oscilação sem alarme falso

Escolha	Motivo
`seq` com bloco de boot	reinício do nó não colide com a chave de deduplicação

D5b. Celular como nó: ponte HTTP → MQTT

A página `/celular` transforma um celular no nó `moinho01`, para demonstrar o fluxo sem hardware e com algo físico na mão. O navegador do celular não abre conexão TCP crua, então não fala MQTT direto. A escolha foi uma **ponte no backend**:

```
celular ──HTTP POST /api/gateway/corrente──▶ backend ──MQTT QoS 1──▶ broker ──▶ assinante ──▶ SQLite
```

Por quê a ponte, e não MQTT sobre WebSocket direto do celular: - não muda o broker nem acrescenta biblioteca externa à página, que fica em HTML/JS puro; - dali em diante o dado segue **o mesmo caminho** do ESP32: mesmo tópico, mesmo payload, mesmo assinante e mesma deduplicação. O backend não tem um "caminho especial do celular"; - o HTTP dá **resposta síncrona**, que o celular mostra como confirmação de recebimento (`← 200 · 4 ms`) — bom para demonstrar envio e recebimento.

O que se perde, e como foi compensado: - sem LWT: o broker não sabe quando o celular some. O STALE por idade do dado (`agora − ts_origem > 5 s`) cobre esse caso, como já cobre o broker fora; - o QoS 1 vale do backend em diante. Entre o celular e o backend, a garantia é a do HTTP: o celular só tira a leitura do buffer com resposta `200`. Se receber `503` (broker fora) ou nenhuma resposta, mantém no buffer e reenvia; - a proteção do celular é a mesma lógica local (D1), rodando no navegador, e continua atuando com o Wi-Fi desligado.

Para demonstrar MQTT ponta a ponta continuam valendo o simulador e o ESP32, que publicam direto no broker. O caminho natural para tirar a ponte depois seria MQTT sobre WebSocket (listener `websockets` no Mosquitto + `mqtt.js` na página).

D6. Fora do escopo desta entrega

Autenticação/TLS no broker, comandos remotos (rearme pela tela), vários dispositivos ao mesmo tempo no dashboard e retenção/limpeza do banco. O contrato (tópicos com `device_id`, `schema` versionado) já permite acrescentar isso depois.

7. Evidências de funcionamento

A evidência mostra o dado **enviado** (log do nó), **recebido** (log do supervisório e banco) e **utilizado** (dashboard), em operação normal e durante uma falha de comunicação. Os testes e os prints foram feitos em 14/10/2026, numa máquina sem Docker, com o broker MQTT em Python (amqtt) no lugar do Mosquitto; o mesmo roteiro com Mosquitto está em `scripts/teste_falha.sh`.

Teste de falha automatizado

O script `scripts/teste_falha_local.ps1` sobe broker, supervisório e simulador, derruba o broker, força uma sobrecorrente sem rede, religa o broker e reenvia leituras duplicadas de propósito. Resultado:

```
== 1/6 subindo broker, backend e simulador
  coletando 12 s de dados normais...
  estado salvo em 01_normal.json
  [OK]    dado atual (nao STALE)
  [OK]    backend gravou 14 leituras

== 2/6 derrubando o broker
  broker parado as 09:29:44

== 3/6 sobrecorrente SEM REDE: a protecao local tem que desarmar
  [OK]    simulador desarmou localmente com o broker fora
  sim| 09:29:50 [PROTECAO] estado=desarmado motivo=sobrecorrente corrente=129.5 A rele=ABERTO (decidido localmente,
SEM REDE)
  estado salvo em 02_degradado.json
  [OK]    backend marcou o dado como STALE (idade 12.5 s)
  [OK]    backend sabe que perdeu o broker
  [OK]    supervisorio ainda NAO sabe do desarme (o dado nao chegou) - por isso o banner
  aguardando mais 15 s com o broker fora (enche o buffer do simulador)...
  sim| 09:30:05 [PROTECAO] sem broker ha 20s - protecao local ATIVA: estado=desarmado corrente=0.0 A | buffer 20/300
descartadas=0
  sim| 09:30:10 [PROTECAO] sem broker ha 25s - protecao local ATIVA: estado=desarmado corrente=0.3 A | buffer 25/300
descartadas=0

== 4/6 religando o broker
  broker de volta as 09:30:15; aguardando o flush do buffer...
  estado salvo em 03_reconstruido.json
  [OK]    dado atual de novo
  [OK]    o desarme feito durante a falha chegou
  [OK]    nenhuma lacuna de seq: historico completo
  [OK]    26 leituras chegaram do buffer com ts_origem/seq originais
  [OK]    desarme registrado com o horario em que aconteceu: 2026-10-14T12:29:50.227Z
  [OK]    historico ordenado por ts_origem
  sim| 09:30:17 [MQTT] conectado ao broker (ficou fora 32.2s); buffer com 31 mensagens para reenviar
  sim| 09:30:17 [BUFFER] flush: 31 mensagens reenviadas com ts_origem/seq originais (restam 0)

== 5/6 idempotencia: reenviando de proposito uma leitura ja gravada
  payload repetido: {"corrente_a":0.2,"motivo":"sobrecorrente","schema":1,"device_id":"moinho01","ts_origem":"2026-
10-14T12:30:19.527Z","seq":1000000049,"estado":"desarmado"}
  [OK]    2 duplicados descartados pela chave (device_id, seq) (0 -> 2)

== 6/6 rearme e encerramento
  [OK]    rearmado: OPERANDO de novo

RESULTADO: todas as verificacoes passaram
```

Logs completos do nó, do supervisório e do broker, e o estado da API em cada etapa, estão no repositório em `evidencias/teste-falha-local-2026-10-14/`.

Log do nó durante a falha

A proteção atua sem rede e o buffer guarda a telemetria; na volta, tudo é reenviado com o `seq` e o `ts_origem` originais.

```
09:32:56 [PROTECAO] estado=desarmado motivo=sobrecorrente corrente=129.8 A rele=ABERTO (decidido localmente, SEM
REDE)
09:32:57 [PROTECAO] sem broker ha 5s - protecao local ATIVA: estado=desarmado corrente=0.1 A | buffer 7/300
descartadas=0
09:33:02 [PROTECAO] sem broker ha 10s - protecao local ATIVA: estado=desarmado corrente=0.3 A | buffer 12/300
descartadas=0
09:33:19 [MQTT] conectado ao broker (ficou fora 26.8s); buffer com 29 mensagens para reenviar
09:33:19 [BUFFER] flush: 29 mensagens reenviadas com ts_origem/seq originais (restam 0)
```

Prints

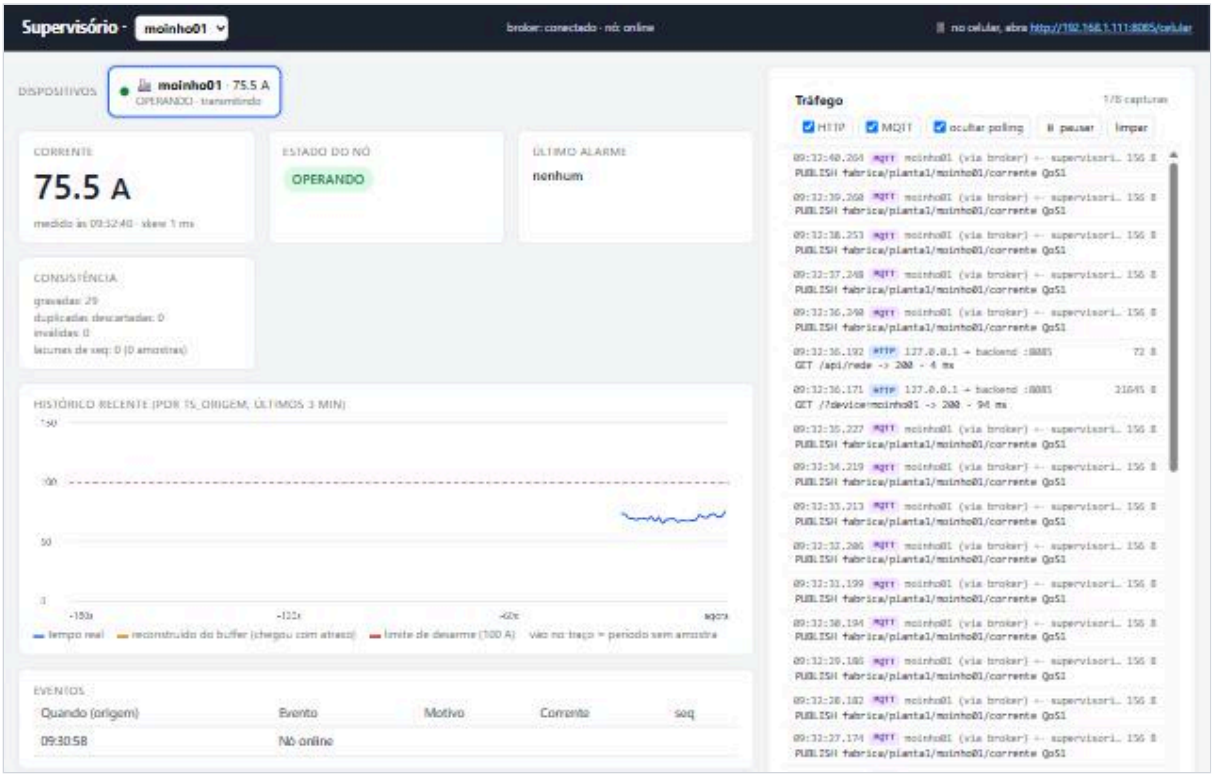


Figura 1. Operação normal: corrente atual, estado OPERANDO, broker conectado, gráfico em tempo real e o painel de tráfego recebendo uma mensagem MQTT por segundo.

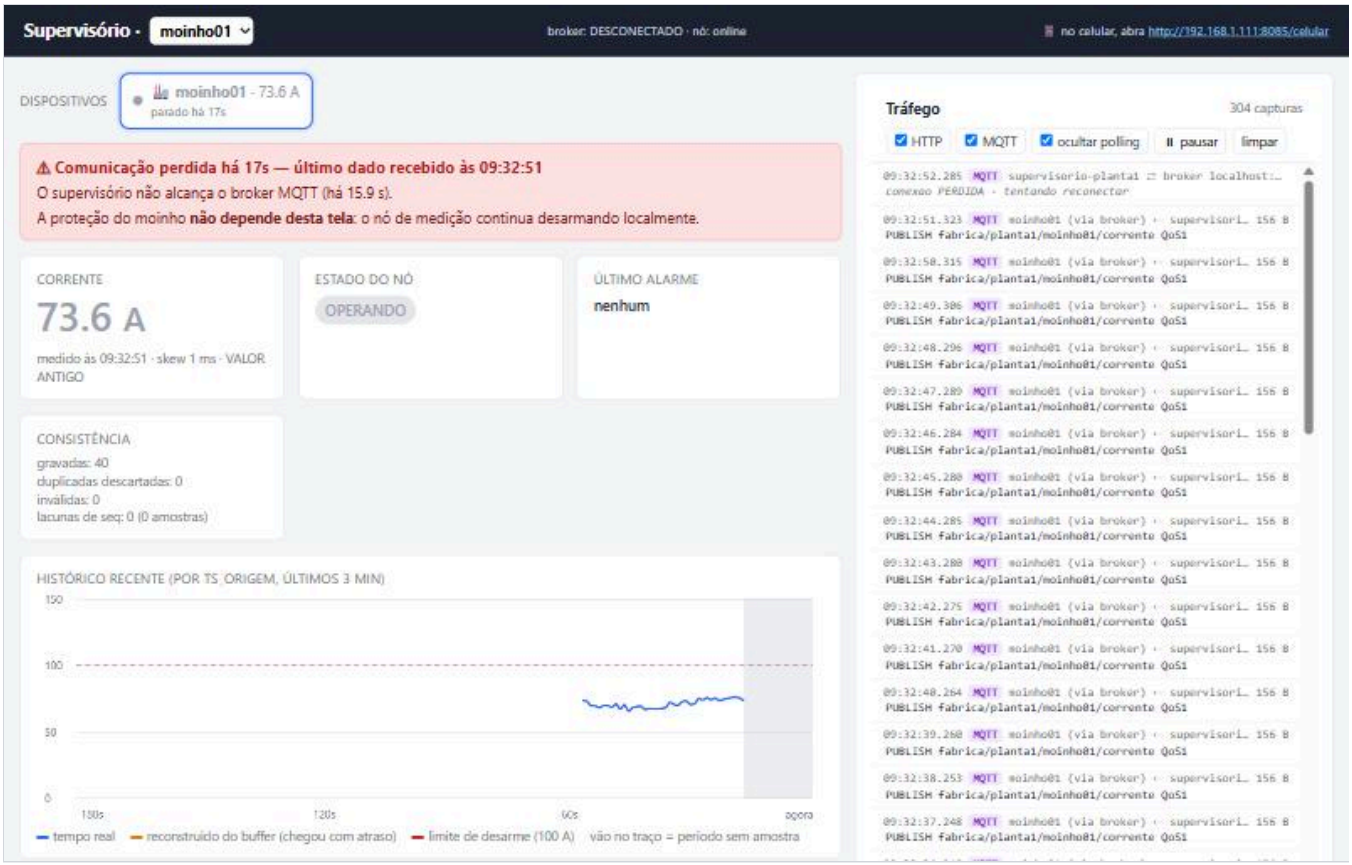


Figura 2. Broker MQTT fora do ar: banner "Comunicação perdida há 17s — último dado recebido às 09:32:51", causa provável, valores em cinza com "VALOR ANTIGO", faixa "sem dados" no gráfico e "conexão PERDIDA" no painel de tráfego. Nesse momento o nó já tinha desarmado sozinho, sem rede.

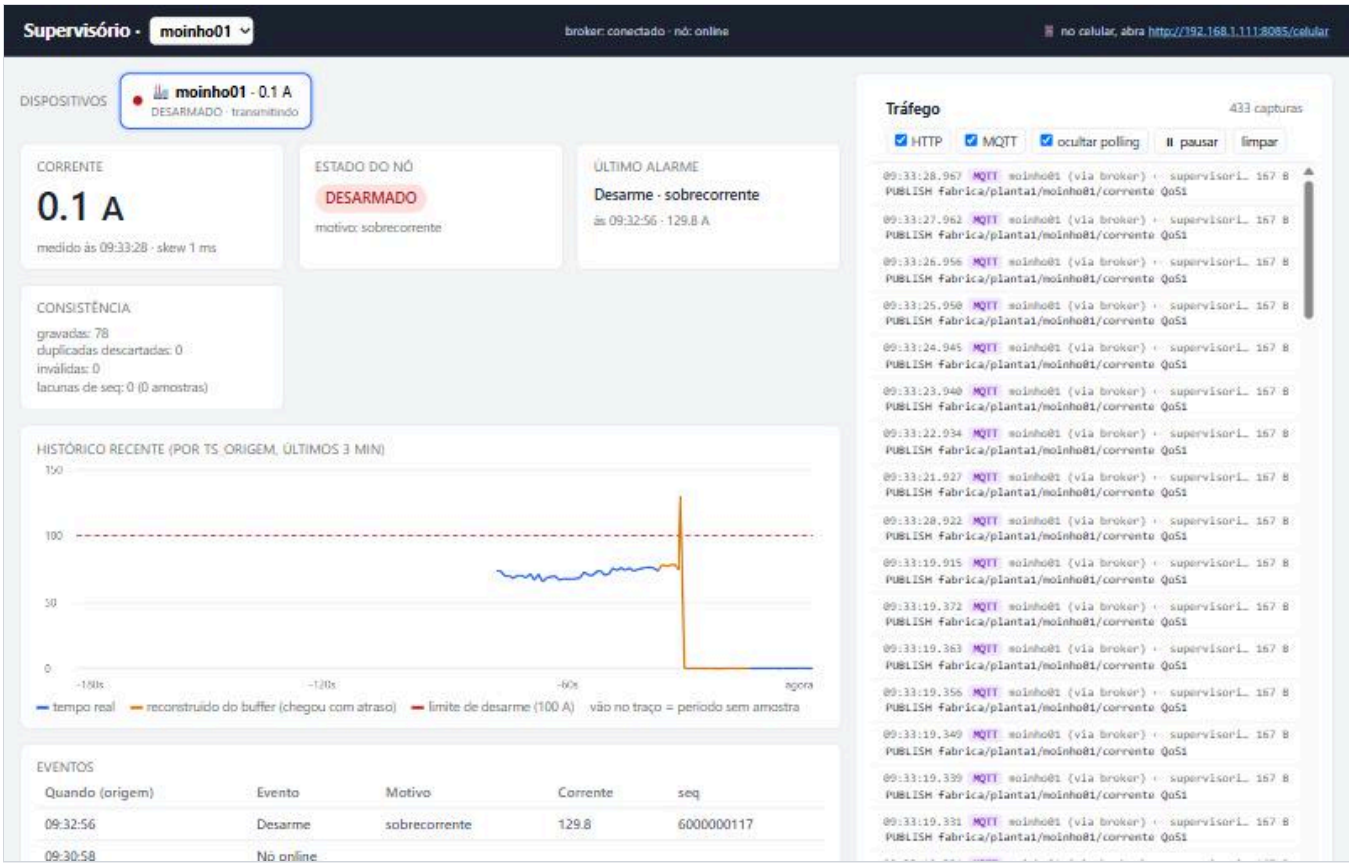


Figura 3. Comunicação de volta: histórico reconstruído (trecho laranja = leituras que vieram do buffer), pico de 129,8 A, estado DESARMADO por sobrecorrente, desarme registrado às 09:32:56 — o horário em que aconteceu, embora só tenha chegado às 09:33:19 — e nenhuma lacuna de seq.



Figura 4. Painel de tráfego com uma mensagem aberta: tópico, QoS 1 e payload com device_id, seq e ts_origem.



Figura 5. Celular fazendo o papel do nó moinho01 (página /celular): carga ajustável, emergência, rearme, falha de sensor e simulação de queda.

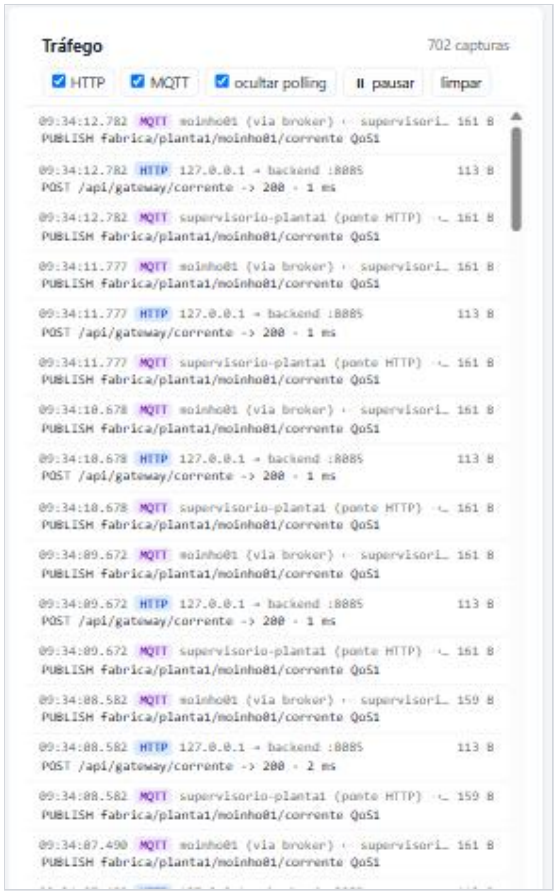


Figura 6. Caminho de cada leitura do celular no painel de tráfego: POST /api/gateway/corrente (HTTP) → PUBLISH da ponte no broker → PUBLISH entregue ao assinante (MQTT).